Kevin Vo - A16215024
10/3/2026
WES 237C

Project 3: Discrete Fourier Transform Report

Project Goal

The goal of this project is to implement the Discrete Fourier Transform (DFT) architecture, which is a common operation in signal processing which calculates a discrete frequency domain representation of the discrete input signal. The input signal being multiplied with cosine and sine waveforms of different frequencies describes how well the input signal correlates with the waveforms.

I. Question 1

Question 1: DFT32 Baseline Implement a baseline HLS implementation for dft_32. You may use the cos() and sin() functions from the math library.

(a) Now think about if you were to use a custom CORDIC algorithm to calculate cos() and sin() (you don't have to implement this). Would changing the accuracy of your CORDIC core make the DFT hardware resource usage change? How would it affect the performance?

Using a custom CORDIC to change the cos() and sin() will directly make the DFT hardware resource change because as shown in the CORDIC assignment, increasing accuracy incurs a heavy resource cost. This will impact performance because it demands more resources and may slow down code execution by increasing the code's latency. This in turn will decrease the throughput of the algorithm. Since CORDIC is supposedly an efficient implementation of the cos() and sin() computation, it may give a slight performance edge over the baseline, but adding more iterations will cause a trade-off between more accuracy in exchange for usage of more resources and longer run-time with no major performance increase.

II. Question 2

Question 2: DFT32 Table Lookup Rewrite the code to eliminate these math function calls (i.e. cos() and sin()) by utilizing a table lookup. Use the provided *_2D.h file.
(a) Make a table that shows the change in resource utilization and performance with between Question 1 and 2.

Table 2.1 shows the change in resource utilization and performance between DFT32 baseline and DFT32 LUT. As shown in the table, the baseline code DFT32 was created based on the sample code written in Figure 4.15 in the textbook, which sequentially calculates the sine and cosine values used in the DFT computation. This implementation has high latency. In Q2, I used the precomputed sine and cosine values coefficients32_2D.h as a lookup table (LUT) and it heavily decreased latency from 54884 in the baseline implementation to just 257 in the LUT implementation. Throughput saw a major increase from 0.0018 MHz to 0.389 MHz. The LUT implementation does in turn use more BRAM, DSP, FF, and LUT than baseline implementation, but it is worth it since it improves performance by a huge margin. As a side note, the question prompt tells us to use a clock period of 10 ns so moving forward all clock periods will be rounded up to that value since it seems that the clock period can change in some questions between 7.256, 8.495, and 9.591 ns, so this was probably mentioned to keep comparisons consistent.

DFT Q1 and Q2: DFT32 Baseline, LUT									
Optimization	**RMSE (R)**	**RMSE (I)**	**Clock (ns)**	**Latency (Cycles)**	**Throughput (MHz)**	**BRAM**	**DSP**	**FF**	**LUT**
Baseline	0.000144	0.000163	10	54884	0.001822024634	2	53	5275	9111
LUT	0.0000022	0.00000183	10	257	0.3891050584	66	640	59208	91730

Table 2.1: Measured values for latency, RMSE, throughput and resource usage of DFT32 Baseline and LUT implementation.

III. Question 3

Question 3: DFT32 Interface Change Modify the DFT function interface so that the input and outputs are stored in separate arrays. Modify the testbench to accommodate this change to DFT interface.

(a) Why did we do this? Does it affect what optimizations you can perform?

Question 3 calls to modify the function interface so that the input and outputs are stored in a separate array. This is so that an interface change optimization can be implemented. In my script, I added two new arrays to store the results of the DFT function, which are real_output[SIZE] and imag_output[SIZE]. For consistency, I renamed the input arrays from real_sample[SIZE] and imag_sample[SIZE] to real_input[SIZE] and imag_input[SIZE]. I also had to dft.h and dft_test.cpp to account for the new arrays. We had to implement interface change because the baseline DFT32 code in the textbook does an “in-place operation” on the input and output data. This means that the results are stored in the same memory location as the input so the output values will overwrite the input values. After implementing this change, it will unlock new optimization avenues such as parallelizing the multiplication and summation operations, which may be hindered by the in-place DFT operation.

(b) Make a table that shows the resource utilization and performance from before and after this change.

Table 3.1 shows the change in resource utilization and performance between DFT32 baseline, LUT, and interface change.

DFT 1, 2, 3: DFT32 Baseline, LUT, and Interface Change									
Optim.	**RMSE (R)**	**RMSE (I)**	**Norm Clock (ns)**	**Latency (Cycles)**	**Throughput (MHz)**	**BRAM**	**DSP**	**FF**	**LUT**
Baseline	0.000144	0.000163	10	54884	0.001822024634	2	53	5275	9111
LUT	0.0000022	0.00000183	10	257	0.3891050584	66	640	59208	91730
iface Δ	0.0000022	0.00000183	10	221	0.4524886878	64	640	59189	91521

Table 3.1: Measured values for latency, RMSE, throughput and resource usage of DFT32 Baseline, LUT, and interface change implementations.

(c) Describe the results you see. Figure 3.1, 3.2, 3.3 and **3.4** all visualize the data in **Table 3.1** in bar graphs to easily compare and deduce trends in the data from.

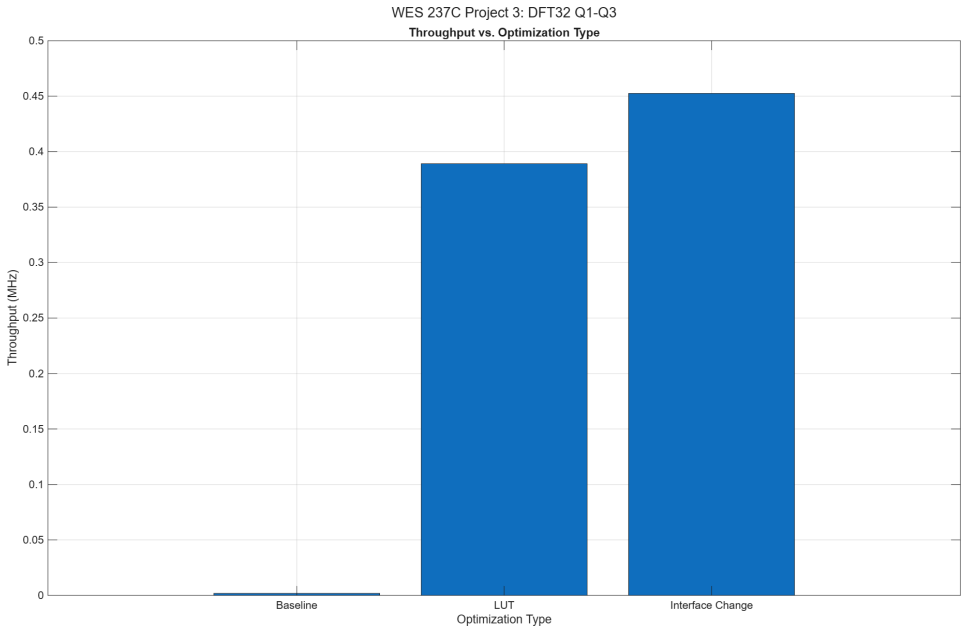


Figure 3.1: Bar graph comparing the calculated throughput in MHz of DFT32 baseline, LUT, and interface change.

Figure 3.1 shows that implementing the LUT optimization in the DFT32 will increase throughput by a major margin, a nearly 21255.64% increase from 0.0018 MHz in the baseline DFT32 to 0.39 MHz in the LUT DFT32. After implementing the interface change optimization, I saw throughput increase by 16.28% from DFT32 at 0.39MHz to 0.45 MHz. This shows that these optimizations were effective.

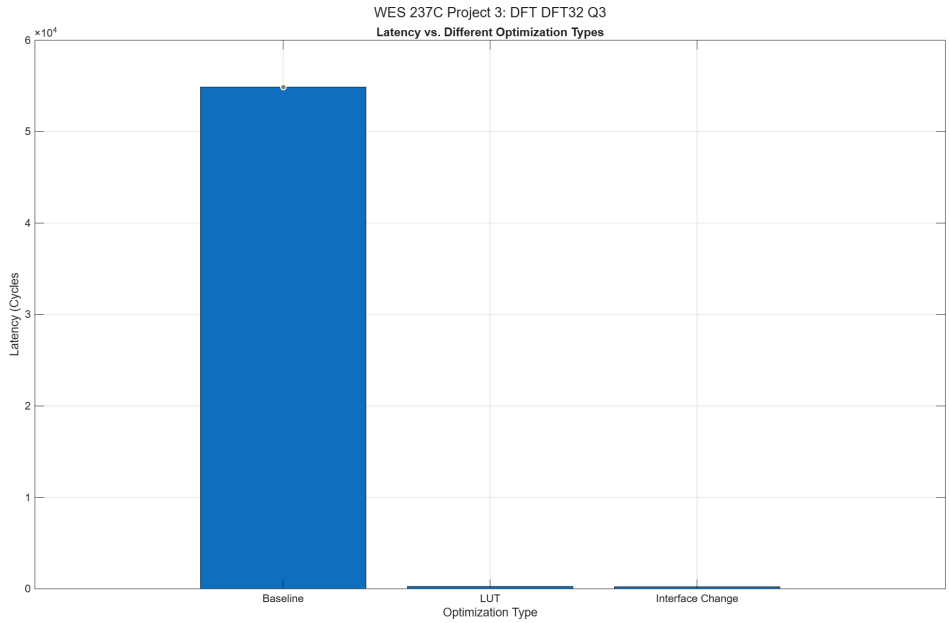


Figure 3.2: Bar graph comparing latency in cycles of DFT32 baseline, LUT, and interface change.

Figure 3.2 shows that implementing the LUT optimization in the DFT32 will decrease latency by 99.53% increase from 54884 cycles in the baseline DFT32 to the LUT DFT32. After implementing the interface change optimization, I saw latency decrease by 14% from DFT32 at 257 cycles to 221 cycles. This shows that these optimizations were effective in reducing latency.

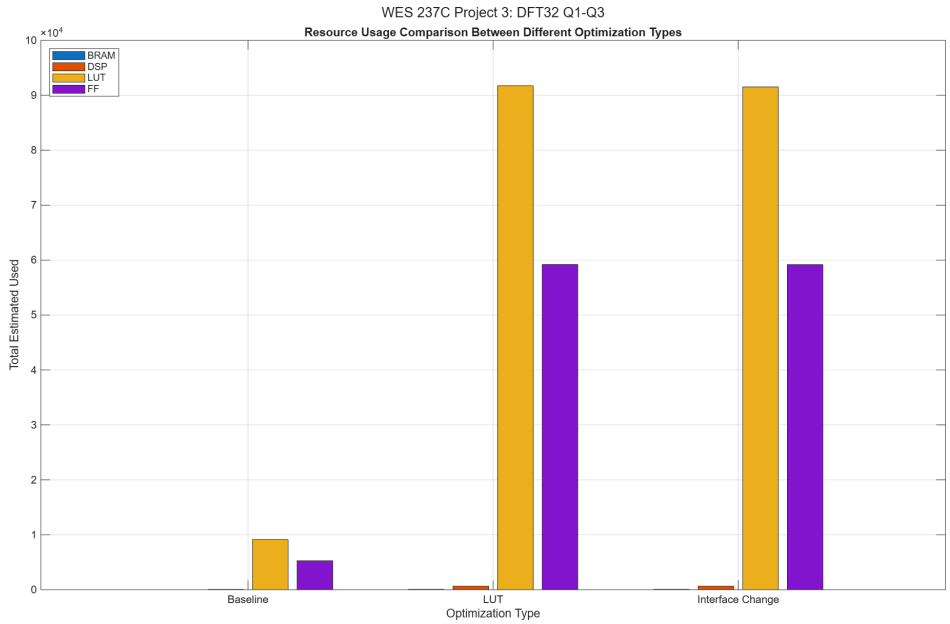


Figure 3.3: Bar graph displaying the measured resource usage of each DFT32 baseline, LUT, and interface change.

Figure 3.3 shows that although the baseline DFT32 implementation uses barely any resources, it has a much lower throughput and higher latency as shown in **Figure 3.1** and **3.2**.

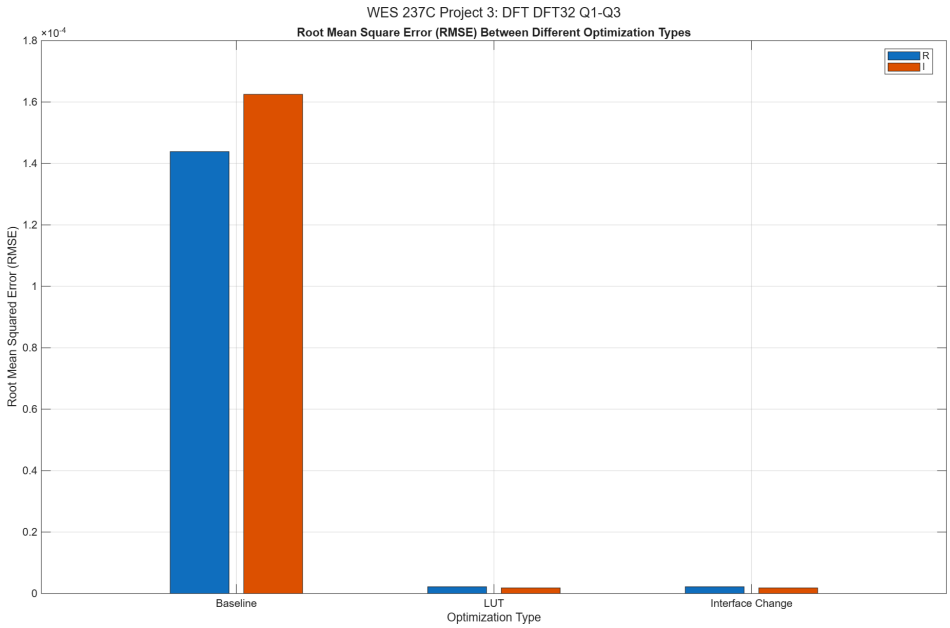


Figure 3.4: Bar graph displaying the measured root mean square error (RMSE) of real and imaginary values for DFT32 baseline, LUT, and interface change.

Figure 3.4 shows that although the baseline DFT32 implementation has a much higher RMSE than the other implementations. This might be due to the fact that it is calculating a sin and cosine value for every iteration so as the amount of computations increased, the value will drift away from what is expected. It makes a lot of sense that implementing a lookup table optimization significantly decreased the amount of measure RSME for real and imaginary values calculated because it reduced the amount of computations needed and thus reduced the amount of possible causes of errors.

IV. Question 4

Question 4: DFT32 Array Partitioning Experiment with array partitioning. Partition all arrays in your implementation. For now, make sure that your loops are being pipelined with a target II of 1.

(a) Use block partitioning. Try factors of 1 (i.e. without partitioning), 2, 4, 8, 16, and 32. Make a table showing the achieved II, resource utilization, and performance of each of these implementations.

Table 4.1 shows the change in resource utilization, achieved interval initialization (II), and performance between DFT32 with Array Partition Factor 1, 2, 4, 8, 16, and 32 optimizations.

DFT 4: DFT Array Partitioning										
Array Part. Fact	**II**	**RMSE (R)**	**RMSE (I)**	**Clock (ns)**	**Latency (Cycles)**	**Throughput (MHz)**	**BRAM**	**DSP**	**FF**	**LUT**
1	1	0.0000022	0.00000183	10	221	0.4524886878	64	640	59189	91521
2	1	0.0000022	0.00000183	10	213	0.4694835681	64	640	59314	91675
4	1	0.0000022	0.00000183	10	209	0.4784688995	64	640	59054	91669
8	1	0.0000022	0.00000183	10	207	0.4830917874	64	640	58540	91705
16	1	0.0000022	0.00000183	10	206	0.4854368932	64	640	57515	91252
32	1	0.0000022	0.00000183	10	203	0.4926108374	64	640	61509	93250

Table 4.1: Measured values for latency, RMSE, II achieved, throughput and resource usage of DFT32 with Array Partition Factor 1, 2, 4, 8, 16, and 32 optimizations.

(b) Plot resource utilization vs the partition factor on one plot.

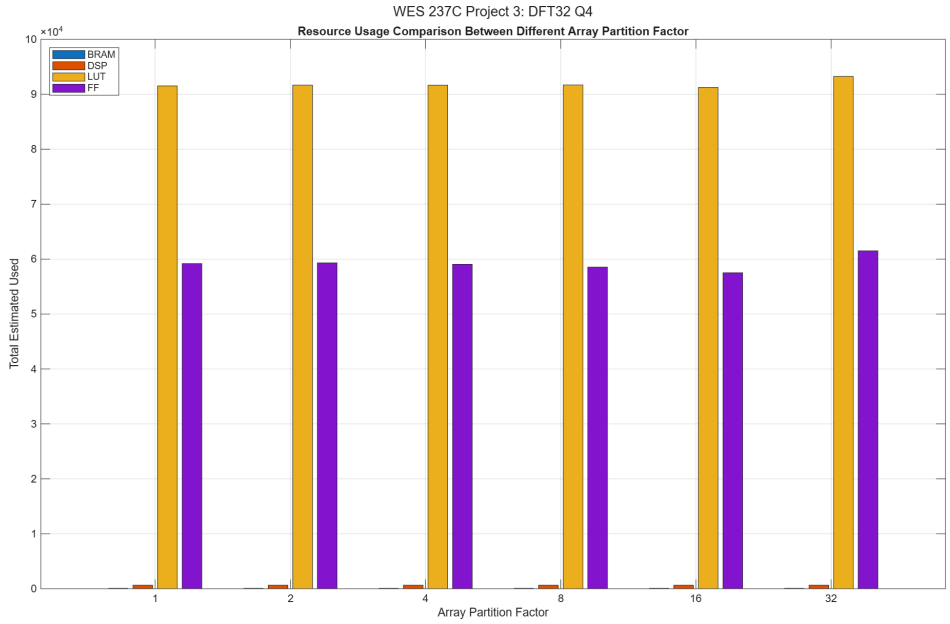


Figure 4.1: Bar graph displaying the measured resource usage of DFT32 with Array Partition Factor 1, 2, 4, 8, 16, and 32 optimizations.

Figure 4.1 shows that changing the array partition factor has minimal impact on the resources used. It seems that as array partition factors increase, the amount of FFs and LUTs increase slightly at each iteration while the BRAM and DSP usage remained mostly the same. In my code, I added array partition factors for the real_input, imag_input, real_output, imag_output variables and the coef_coeff_table and sin_coeff_table variables at dimension 2 since they are 2D arrays.

(c) Plot throughput & latency vs the partition factor on separate plots.

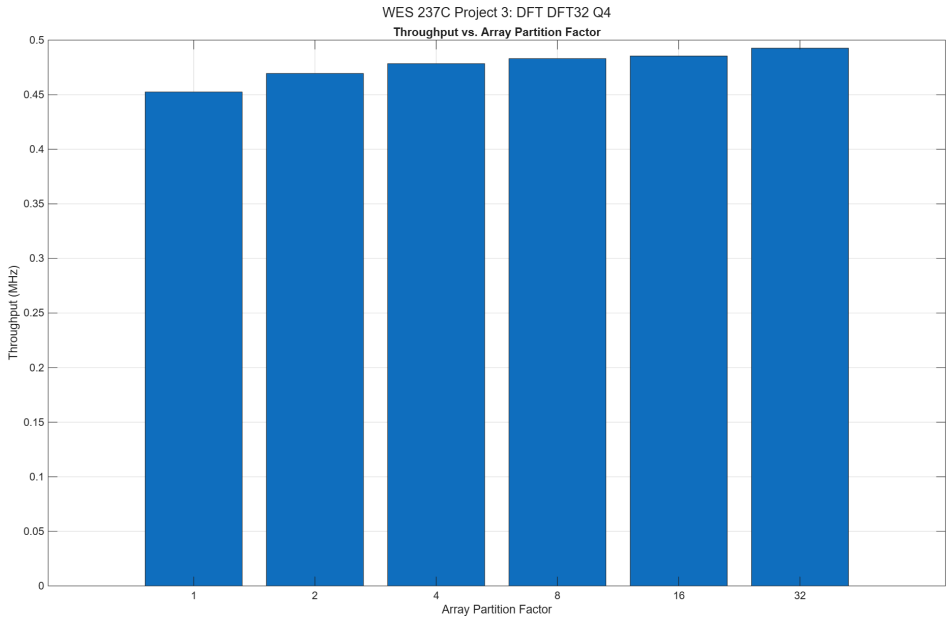


Figure 4.2: Bar graph comparing the calculated throughput in MHz of DFT32 with Array Partition Factor 1, 2, 4, 8, 16, and 32 optimizations.

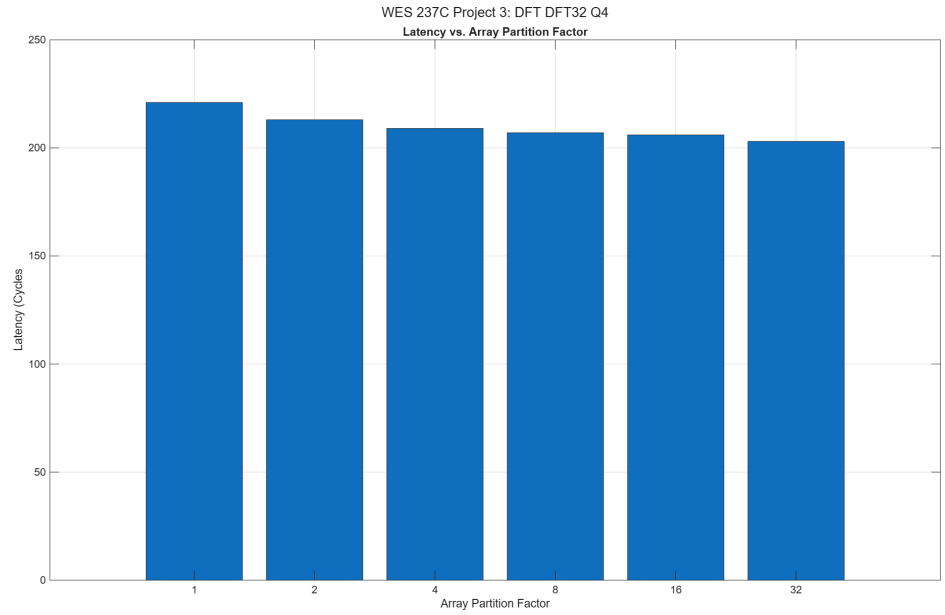


Figure 4.3: Bar graph comparing the latency in cycles of DFT32 with Array Partition Factor 1, 2, 4, 8, 16, and 32 optimizations.

Figure 4.2 and **4.3** shows that there is a slight improvement in throughput as you increase the array partition factor and slight decrease in latency. Might need other optimization methods to take advantage of array partitioning.

V. Question 5

Question 5: DFT32 Loop Unrolling Experiment with loop unrolling. Unroll the inner loop only. Use the best array partitioning from Question 4. In order to prevent automatic unrolling by Vitis HLS, you will need to remove the #pragma HLS PIPELINE pragmas from both loops. Double check that your explicit unrolling is being applied by looking at the synthesis report. (a) Try factors of 1 (i.e. without unrolling), 2, 4, 8, 16, and 32. Make a table showing the achieved II, resource utilization, and performance of each of these implementations.

Table 5.1 shows the change in resource utilization, achieved interval initialization (II), and performance between DFT32 with Loop Unrolling factor 1, 2, 4, 8, 16, and 32 optimizations.

Table 5.1 - DFT 5: DFT32 Loop Unrolling

Unroll Factor	II	RMSE (R)	RMSE (I)	Clock (ns)	Latency (Cycles)	Throughput (MHz)	BRAM	DSP	FF	LUT
1	1	0.0000022	0.00000183	10	6721	0.01487873828	32	5	847	1314
2	1	0.0000022	0.00000183	10	6210	0.01610305958	64	5	5164	1627
4	1	0.0000022	0.00000183	10	5954	0.01679543164	64	7	5696	2301
8	1	0.0000022	0.00000183	10	5826	0.01716443529	64	7	6228	2419
16	1	0.0000022	0.00000183	10	5762	0.01735508504	64	7	8476	4345
32	1	0.0000022	0.00000183	10	206	0.4854368932	64	640	57515	91252

(b) Plot resource utilization vs the partition factor on one plot.

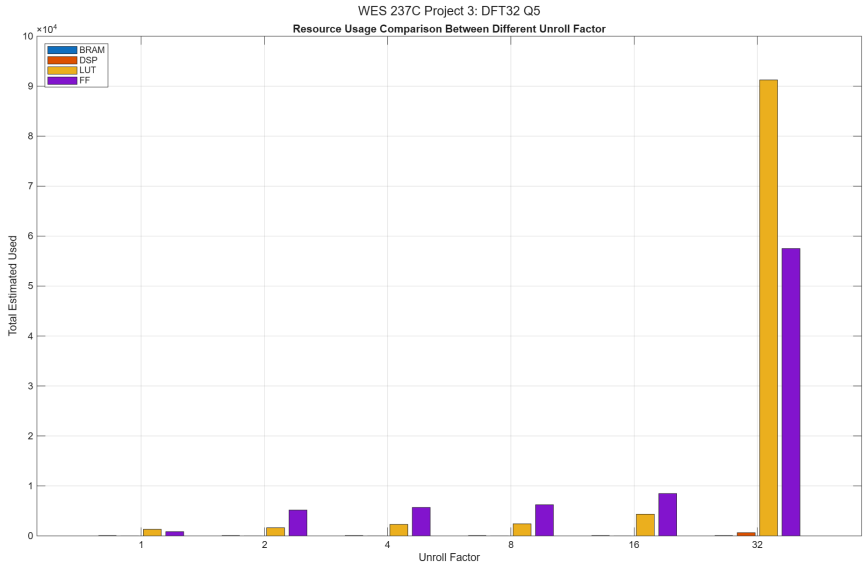


Figure 5.1: Bar graph displaying the measured resource usage of DFT32 with Loop unrolling factor 1, 2, 4, 8, 16, and 32 optimizations.

Figure 5.1 shows that changing the loop unrolling factor has minimal consumption at 1, 2, 4, 8, and 16 but at loop unrolling factor of 32 the amount of resources used increased dramatically. This makes a lot of sense because since I added the unroll factor of 32 in the innermost loop, it does a complete unrolling of the read, multiplication, and add and subtraction operations in the inner loop with parallel hardware. The lower unrolling factor only partially unrolls the loop so that is why the resource usage increase is relatively low until factor of 32 since there is not enough parallelization to fully utilize it.

(c) Plot throughput & latency vs the partition factor on separate plots.

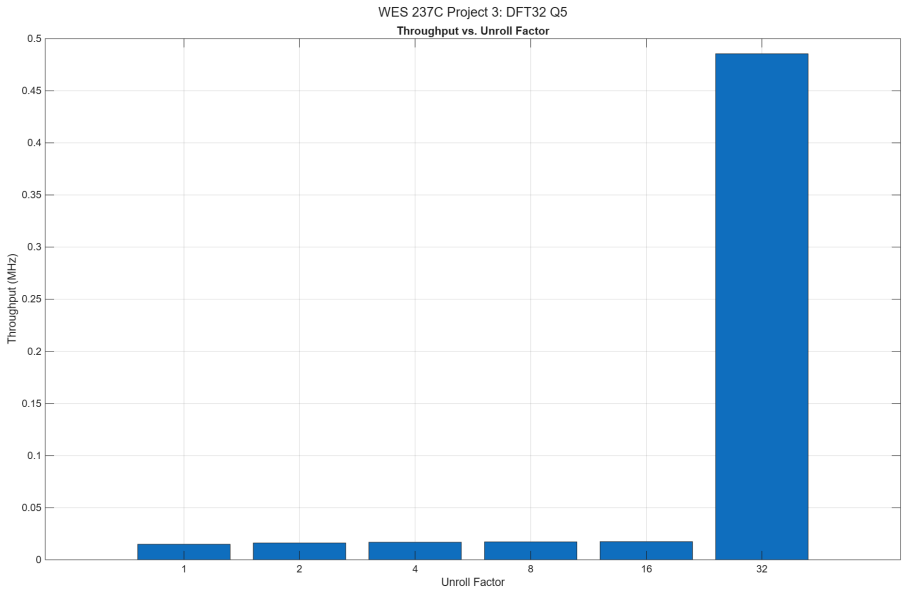


Figure 5.2: Bar graph comparing the calculated throughput in MHz of DFT32 withLoop Unroll Factor 1, 2, 4, 8, 16, and 32 optimizations.

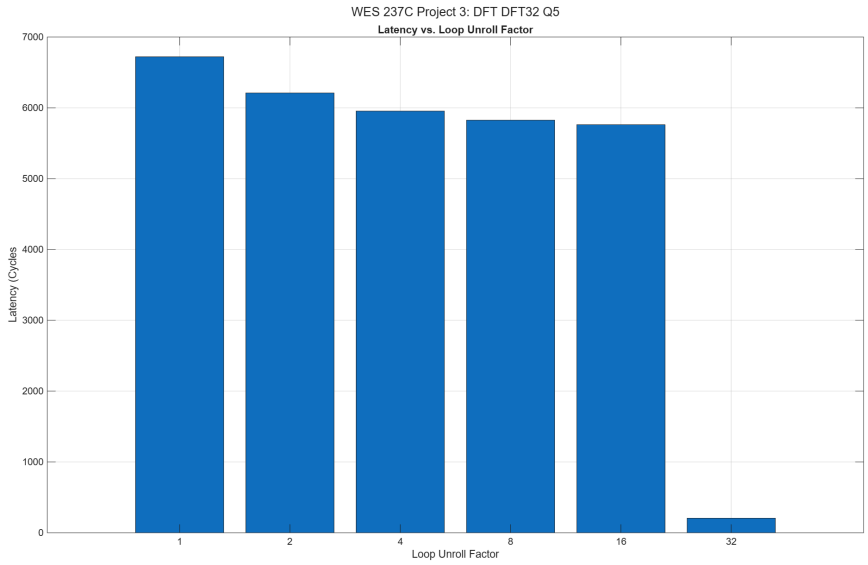


Figure 5.3: Bar graph comparing the latency in cycles of DFT32 with Loop Unroll Factor 1, 2, 4, 8, 16, and 32 optimizations.

Figure 5.2 and **5.3** shows that there is a significant improvement in throughput as you increase the loop unroll factor and major decrease in latency. For instance, going from a loop unroll factor of 16 with throughput of 0.01735 MHz to factor of 32 with throughput of 0.4854 MHz allows for a 2697.69% or 28 times increase in throughput. I also see a similar trend in latency decrease. Latency decreased from 5762 cycles to only 206 cycles going from a loop unroll factor of 16 to 32, or a 96.425% decrease.

VI. Question 6

Question 6: DFT1024 Baseline You should refer to the baseline DFT code at Figure 4.15 of the textbook.

(a) Write a baseline DFT1024 using the sin() and cos() math functions. Do not apply any HLS pragmas. Report latency, throughput, and resource utilization.

The baseline DFT1024 dft.cpp script that I made for Question 6A was created based on the Baseline DFT32 one from Q1. The main difference is that it was designed already with the output array interface interchange feature already implemented so I went with the variables already there. Something I noticed was the RMSE would be too great that I would not pass the dft_test.cpp checks so I changed all the data types used in computation from floats to doubles and it fixed the issue. No HLS pragmas were added. **Table 6.1** shows the measured latency, calculated throughput, and resource utilization of the baseline DFT1024.

DFT 6a: DFT1024 Baseline									
Optimization	**RMSE (R)**	**RMSE (I)**	**Clock (ns)**	**Latency (Cycles)**	**Throughput (MHz)**	**BRAM**	**DSP**	**FF**	**LUT**
Baseline	0.002134	0.00899179	10	96485377	0.000001036426484	16	220	14885	17789

Table 6.1: Measured values for latency, RMSE, throughput and resource usage of DFT1024 Baseline.

(b) A full 2D lookup table no longer fits on a PYNQ-Z2 board. We can only store a 1D array of precomputed sin() and cos() values. Re-write the baseline DFT1024 to use the pre-computed values. Report the latency, throughput and resource utilization.

The LUT DFT1024 dft.cpp script that I made for Question 6A was created based on the LUT DFT32 one from Q2. The implementation draws the sin() and cos() values from the coefficients1024.h file. I had a similar issue with the float values causing RMSE values to be too great so I changed all data types being used in computation to be double instead of floats. **Table 6.2** shows the measured latency, calculated throughput, and resource utilization of the LUT DFT1024.

DFT 6b: DFT1024 LUT									
Optimization	**RMSE (R)**	**RMSE (I)**	**Clock (ns)**	**Latency (Cycles)**	**Throughput (MHz)**	**BRAM**	**DSP**	**FF**	**LUT**
LUT	0.002138	0.00800072	10	8388634	0.00001192089201	4	4	2499	2536

Table 6.2: Measured values for latency, RMSE, throughput and resource usage of DFT1024 LUT.

VII. Question 7

Question 7: DFT1024 Loop optimization The baseline DFT1024 from Figure 4.15 of the textbook has data dependencies in the inside loop, which could limit parallelism. One way to tackle this issue is to interchange the two loops. Implement this change and report the latency, throughput, and resource.

For this question I used Q6b's code as a basis for Q7. In the code, I interchanged the outer frequency loop, which iterates through i, with the inner time-domain summation loop, which iterates through j. This means that in Q7, the outer loop will iterate through j and the inner loop will iterate through i. The reason I did this was because it means that it allows for better use of resources. This is because having j in the inner loop makes it constantly refetch data from memory for variables that cause unnecessary latency. **Table 7.1** shows the measured latency, calculated throughput, and resource utilization of the LUT DFT1024.

DFT 7: DFT1024 Loop Optimization									
Optimization	**RMSE (R)**	**RMSE (I)**	**Clock (ns)**	**Latency (Cycles)**	**Throughput (MHz)**	**BRAM**	**DSP**	**FF**	**LUT**
Loop Optimization	0.002138	0.00800072	10	1050664	0.00009517790654	12	56	5563	7574

Table 7.1: Measured values for latency, RMSE, throughput and resource usage of DFT1024 Baseline.

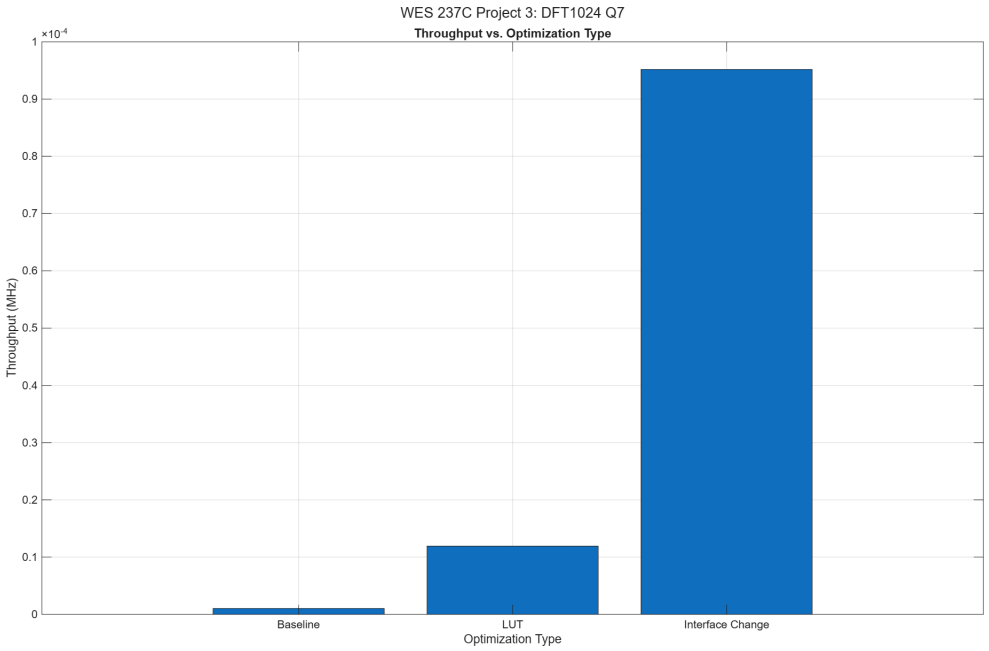


Figure 7.1: Bar graph comparing the calculated throughput in MHz of DFT1024 Baseline, LUT, and Interface Change.

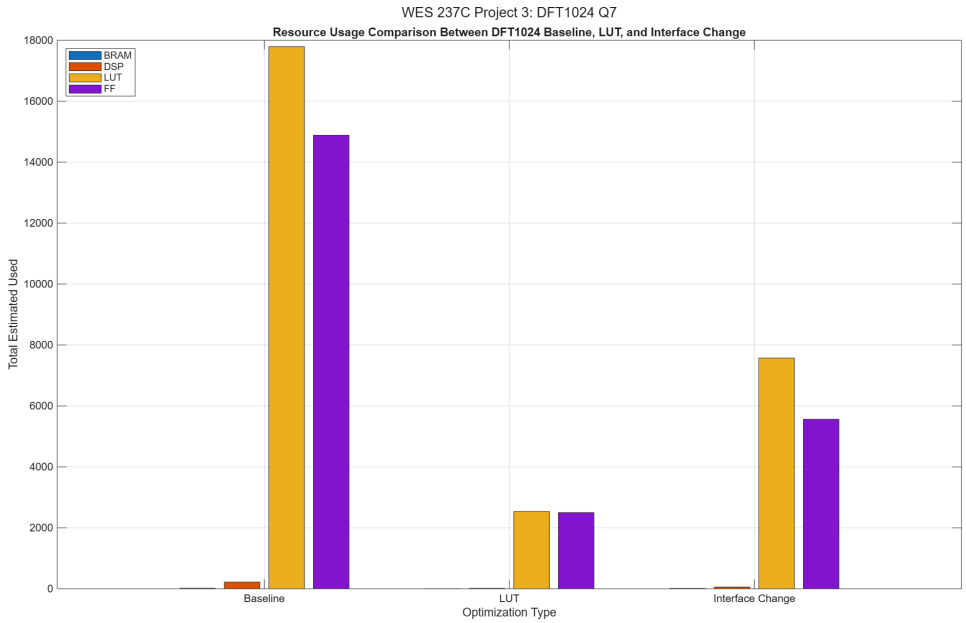


Figure 7.2: Bar graph comparing the measured resource usage of DFT1024 Baseline, LUT, and Interface Change.

Figure 7.1 and **7.2** shows the measured latency, calculated throughput, and resource utilization of the Baseline, LUT, and Interface Change DFT1024. As shown, it significantly increases the throughput. The Interface Interchange DFT1024 implementation in Q7 at 0.00009517790654 MHz is a 698.41% increase, or about an 8 times increase, from 0.00001192089201 MHz there in Q6b. The resource usage is more efficient than the baseline and Q6b DFT1024 since some arrays are only read in once rather than constantly.

VIII. Question 8

Question 8: DFT1024 Best Design: Now that you have explored different optimizations for DFT32, we can go ahead and try their ideas on DFT1024.

(a) Try any optimization techniques and describe your methodology.

To find the best combination of the optimization techniques for DFT1024, I utilized the loop interchange script I wrote for Q7 as a basis since that was the best dft.cpp script I wrote that had the highest throughput and best resource usage according to **Figure 7.1** and **7.2**. I then revisited the plots I wrote in the previous section to see what other kind of optimization techniques I can implement based on what was best for that question. The Q7 code already implemented the best optimization technique for Q3, which is storing the real and imaginary input and output variables in separate arrays (total 4) so I kept that. Since Array Partitioning did not show much promise in Q4 according to my results in Table 4.1 with its minimal improvements and 0 change in resource usage, I did not really try to incorporate it in my best optimization. I did try, but I got an error running it so I gave up.

Moving on to Q5, I saw that adding a loop unrolling factor did significantly improve throughput, reduce latency as shown in **Table 5.1** and **Figures 5.1** and **5.2** so I implemented it in my attempt for the

best optimization technique. However, as warned the resource usage was so high that it exceeded 100% for all categories. For instance, U32 had 114% BRAM, 814% DSP, 147% FF, and 430% LUT usage, which means it cannot fit on a PYNQ-Z2 board. After this finding, I would gradually reduce the loop unrolling factor from 32 to 16, 16 to 4, 4 to 3, and 3 to 2. I did experiment with changing the II from 1 to 2 for U16 and U4, but I found that did not help.

(b) Report the latency, throughput, and resource utilization of your best design. Your design must fit on the PYNQ-Z2 board, which mean all resource utilizations must be less than 100%.

Warning: Avoid aggressively unrolling loops, partitioning arrays, or pipelining. This could lead to long design synthesis time, and your design may not fit on board.

Table 8.1 summarizes the different optimization strategies I explored to try to find the best DFT1024 design which not only had a high throughput and low latency, but a design whose resource usage could fit on a PYNQ-Z2 board, which was the ultimate limitation. All of these runs ran successfully and relatively quickly so I did not run into a long synthesis time unless I ran into a syntax error or when I had issues with make tests where the output would be either an error or RMSE was too high (this problem was solved back in Q6a). Ultimately, the best design I found is highlighted in green below which was the base code for Q7, which implements HLS PIPELINE = 1 in all loops, loop unrolling factor = 3 and the interface change optimizations from adding a separate output array in Q3 and interchanging loops in Q7. This design was the highest throughput, and low latency compared to the others and can be run on a PYNQ-Z2 board. I will explain more details below in the later sections.

DFT 8: DFT1024 Loop Optimization									
Optimization	**RMSE (R)**	**RMSE (I)**	**Clock (ns)**	**Latency (Cycles)**	**Throughput (MHz)**	**BRAM**	**DSP**	**FF**	**LUT**
Baseline	0.002134	0.00899179	10	96485377	0.000001036426484	16	220	14885	17789
LUT	0.002138	0.00800072	10	8388634	0.00001192089201	4	14	2499	2536
Interface Change	0.002138	0.00800072	10	1050664	0.00009517790654	12	56	5563	7574
Q7 + loop unroll 2	0.002138	0.00800072	10	526376	0.0001899782665	20	112	10562	14826
Q7 + loop unroll 3	0.002138	0.00800072	10	352297	0.0002838514095	32	171	15807	22213
Q7 + loop unroll 4	0.002138	0.00800072	10	527401	0.0001896090451	20	112	12635	15796
Q7 + loop unroll 16	0.002138	0.00800072	10	67627	0.001478699336	160	220	106k	115k
Q7 + loop unroll 16	0.002138	0.00800072	10	134187	0.0007452286734	80	448	147566	60841
Q7 + loop unroll 32	0.002138	0.00800072	10	34863	0.002868370479	320	814	157k	229k

Table 8.1: Measured values for latency, RMSE, II achieved, throughput and resource usage of DFT1024 iterations try to find the best optimization. This table compares modifications of the script in Q7 with loop unrolling 32 (U32), Loop unrolling 16 with II = 2 (U16II2), loop unrolling 16, loop unrolling 4 with II = 2, loop unrolling 3 (U3), loop unrolling 2 (U2) against the original Baseline, LUT, and interface change optimizations for Q7.

Figure 8.1, 8.2, and **8.3** better visualizes the different design iterations in Table 8.1.

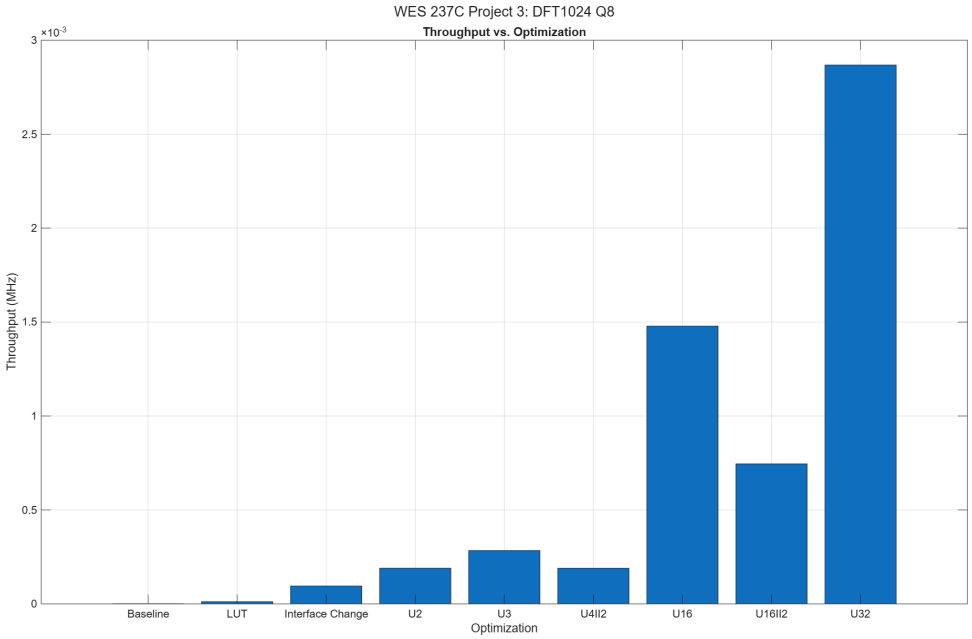


Figure 8.1: Bar graph comparing the calculated throughput in MHz of DFT1024 to try to find the best optimization. This plot compares modifications of the script in Q7 with loop unrolling 32 (U32), Loop unrolling 16 with II = 2 (U16II2), loop unrolling 16, loop unrolling 4 with II = 2, loop unrolling 3 (U3), loop unrolling 2 (U2) against the original Baseline, LUT, and interface change optimizations for Q7.

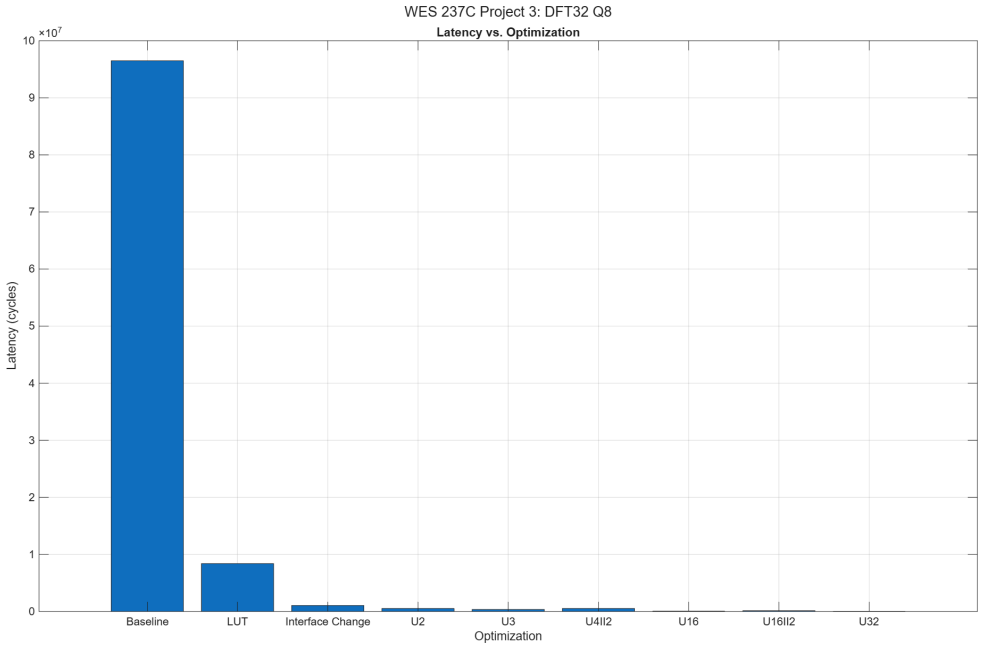


Figure 8.2: Bar graph comparing the measured latency in cycles of DFT1024 to try to find the best optimization. This plot compares modifications of the script in Q7 with loop unrolling 32 (U32), Loop unrolling 16 with II = 2 (U16II2), loop unrolling 16, loop unrolling 4 with II = 2, loop unrolling 3 (U3), loop unrolling 2 (U2) against the original Baseline, LUT, and interface change optimizations for Q7.

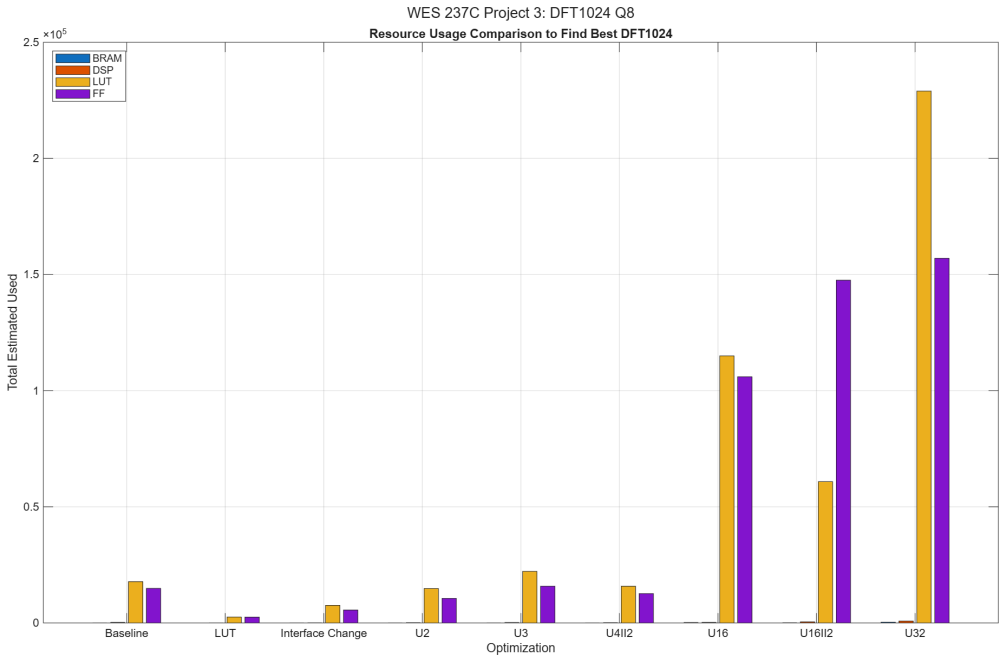


Figure 8.3: Bar graph comparing the measured resource utilization of DFT1024 to try to find the best optimization. This plot compares modifications of the script in Q7 with loop unrolling 32 (U32), Loop unrolling 16 with II = 2 (U16II2), loop unrolling 16, loop unrolling 4 with II = 2, loop unrolling 3 (U3), loop unrolling 2 (U2) against the original Baseline, LUT, and interface change optimizations for Q7.

Using Figure **8.1-8.3** as a guide, I settled on Q7 + loop unrolling 3 as the best design that incorporates as much optimization as I was able to add based on the previous problems. It was the design with the highest throughput, lowest latency, and resource usage compatible with a PYNQ Z2 Board. The BRAM usage of this implementation is only 11%, 77%, 14%, and 41%, which makes it lower than the limitation given in the question. The U4 II=2 is slightly lower in resource utilization, but the throughput is too high. Before I tried U4 II=2, I tried U4, which is not in the plots. I exceeded the DSP by 101%, so I didn't use that. I unfortunately did not plot this since I made a quick change to test II=2 and forgot to save my results from that run. Since I did not want to take any risks, the final code ended up being Q7 + loop unroll factor of 3 and this would be the design that I will use for Question 9 streaming portion.

IX. Question 9

Question 9: Streaming Interface Synthesis: Modify your design to allow for streaming inputs and outputs using hls::stream. You must write your own testbench to account for the function interface change from DTYPE to proper hls::stream. You can learn about hls::stream from the HLS Stream Library. You should also follow the Lab: Axistream Multiple DMAs example. Report the latency, throughput, and resource utilization of your design. Resource utilization must be under 100%. Using

your optimized DFT1024 is optional, you can also just convert the baseline DFT1024 to a streaming interface.

Table 9.1 shows the measured latency, calculated throughput, and resource utilization of the best DFT1024 implementation from Q8 but adapted for streaming interface. This code would end up being used to create the bitstream for the PYNQ demo.

DFT Q9: DFT1024 Streaming Interface									
Optimization	**RMSE (R)**	**RMSE (I)**	**Clock (ns)**	**Latency (Cycles)**	**Throughput (MHz)**	**BRAM**	**DSP**	**FF**	**LUT**
Q8 Streaming	0.00213822	0.008000728	10	385035	0.3891050584	66	640	59208	91730

Table 9.1: Measured values for latency, RMSE, throughput and resource usage of DFT32 Baseline and LUT implementation.

X. PYNQ Demo

Figure 9.1 shows the implemented IP Integrator block design of the DFT1024 from Q9 through Vivado and **Figure 9.2** is the Vivado device view of it. A bitstream was generated from this which was ran on the Xilinx PYNQ-Z2 board in the video. Sample outputs from this bitstream can be seen in Figure 9.3 and 9.4. The video of the demo can also be found below in the link at the bottom of this section.

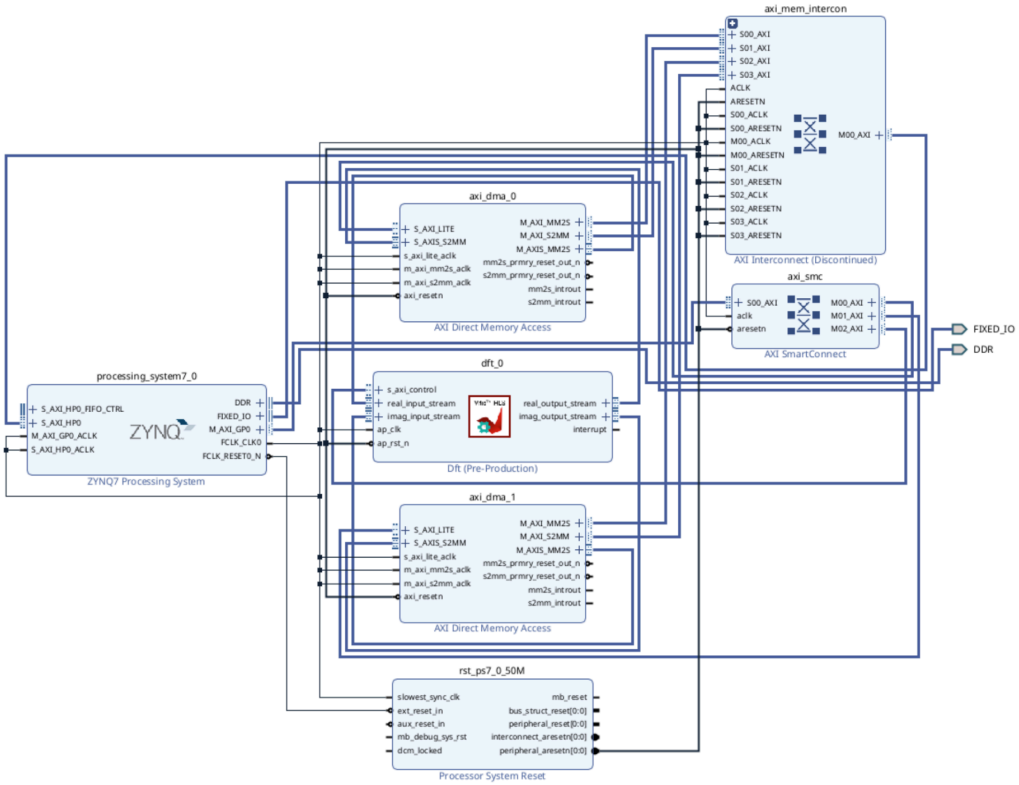


Figure 9.1: Vivado DFT1024 implemented device integrator block from Q9.

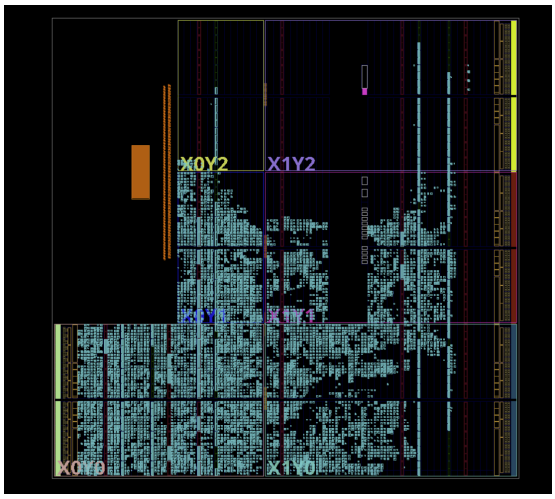


Figure 9.2: Vivado device view of DFT1024 implementation from Q9.

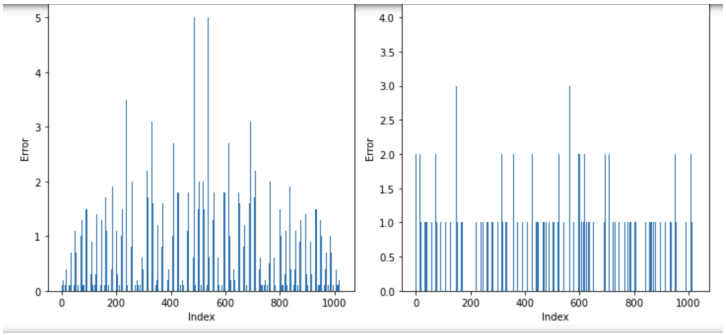


Figure 9.3. Real and imaginary error plot output from PYNQ Demo

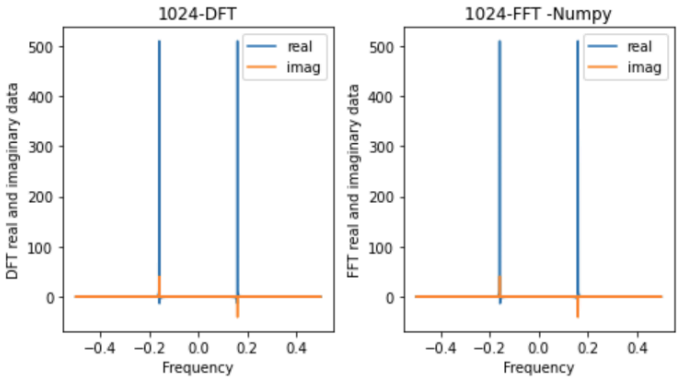


Figure 9.3: DFT and FFT plots of real and imaginary data from PYNQ demo.

DFT PYNQ Board Demo: https://youtu.be/tpaweKuQ-58

XI. Code Submission

The code for this project can be found through the link on my personal Github with username KVo5657:
https://github.com/KVo5657/wes237c_submission/tree/main/dft
Classroom50: https://github.com/UCSD-237C-2026/wes-237c-project-3-dft-group-12/tree/main/dft